BRAIN-INSPIRED MULTI-TIMESCALE MEMORY FOR LANGUAGE AGENTS

Anonymous authors
Manuscript in preparation

ABSTRACT

Long-lived language agents must remember not only what was mentioned, but who experienced it, when it was true, and how it should guide a later decision. Retrieving a relevant memory does not ensure that these distinctions survive abstraction, and revising an individual fact does not ensure that its derived knowledge is revised with it. We propose Multi-Timescale Memory (MTM), a brain-inspired framework that separates rapid episodic encoding, query-time evidence binding, and slower semantic and procedural consolidation. Its central mechanism is evidence-bound reconsolidation: each derived memory and each use of that memory retain explicit dependencies, so a supported correction can invalidate and rebuild affected knowledge without rewriting unrelated history. A typed router assembles episodic evidence, semantic claims, and applicable procedures into a bounded working context; conflict-aware fusion preserves temporal scope and unresolved uncertainty. We ground the design in a verified LoCoMo example and specify a compact evaluation on LoCoMo and LongMemEval of accuracy and cost, component contributions, and update-aware reasoning. The framework and algorithm are proposed here; benchmark performance and superiority over prior systems remain to be established experimentally.

1 INTRODUCTION

An agent can remember every important noun and still fail to remember an event. In a LoCoMo conversation, Caroline recommends *Becoming Nicole* in July. Three months later, Melanie says that she has been reading "that book" recommended earlier. Answering which book Melanie read requires binding the recommendation, its participants, and the later reading event. A list of book titles is insufficient; an isolated summary of the later session is also insufficient. The required memory is a relationship across episodes, grounded in their sources rather than reconstructed from plausible associations.

This difficulty extends beyond retrieval. Once the agent converts experiences into a general claim or a reusable procedure, later evidence may change the conditions under which that abstraction is valid. Correcting a source fact without revisiting its dependent summaries can leave stale knowledge active. Conversely, replacing all old information with the latest statement destroys historical distinctions. Useful memory therefore requires coordinated *encoding, binding, abstraction, and revision*, not simply a larger store.

Existing research already addresses important parts of this problem. Associative retrieval connects distant evidence; temporal graphs represent evolving facts; agentic notes revise linked memories; and sleep-time updates reduce online maintenance overhead (Gutiérrez et al., 2024; Rasmussen et al., 2025; Xu et al., 2025; Fang et al., 2026). Reflection and memory-utility learning also make feedback part of memory management (Shinn et al., 2023; Latimer et al., 2025; Zhang et al., 2026b). We do not claim that these systems lack structure, temporal reasoning, or feedback. The question we isolate is more specific: *can the path from source experience to abstraction to use also serve as the path along which a correction propagates?*

Our framework draws a functional analogy to complementary memory processes. Rapid encoding preserves individual experiences, while slower integration constructs reusable knowledge (McClelland et al., 1995; Kumaran et al., 2016). Retrieval brings relevant information into an active context, and subsequent revision can be restricted to the reactivated knowledge. This is an engineering

interpretation, not an anatomical model or a claim that an LLM reproduces human cognition. Brain-inspired staging alone is not a new contribution: HippoRAG and LightMem explicitly adopt related motivations.

MTM instantiates the analogy with three long-term memory types and three update timescales. Episodic memory stores observations; semantic memory stores source-backed claims; procedural memory stores conditional retrieval or action strategies. A shared dependency graph binds these representations. At inference time, the agent retrieves and checks evidence before answering. At revision time, observable corrections identify a dependency-local repair set. At background consolidation time, replay rebuilds affected abstractions and validates reusable procedures. An answer generated by the agent is never, by itself, evidence that the answer is correct.

The intended contribution is a concrete *evidence-bound reconsolidation* algorithm, rather than a collection of brain-named modules. We specify its record types, routing rule, conflict semantics, update boundaries, and failure behavior. A source-verified case illustrates why relational binding matters. The evaluation uses two conversational benchmarks to test overall utility at a common budget, isolate component contributions, and examine relational and update-aware reasoning. The empirical claims remain open until these tests are run.

2 Related Work

Foundations of agent memory. RAG complements parametric knowledge with an external corpus (Lewis et al., 2020). Early agent systems extend external memory to interaction histories through experience streams and summaries (Park et al., 2023; Zhong et al., 2024), explicit context hierarchies (Packer et al., 2023), verbal feedback (Shinn et al., 2023), and reusable executable skills (Wang et al., 2024). These approaches establish complementary forms of persistent state: factual records, compressed context, and experience that can guide subsequent decisions.

Representation, maintenance, and efficient access. Later systems organize memory as linked notes or extracted facts (Xu et al., 2025; Chhikara et al., 2025), associative and temporal graphs (Gutiérrez et al., 2024; 2025; Rasmussen et al., 2025; Jiang et al., 2026), and hierarchical or heterogeneous stores (Rezazadeh et al., 2025; Kang et al., 2025; Li et al., 2025). Hindsight further distinguishes evidence from synthesized summaries and beliefs (Latimer et al., 2025). Efficiency-oriented approaches combine compression, selective access, and consolidation (Lee et al., 2024; Fang et al., 2026; Liu et al., 2026); recent systems additionally adapt memory types and structure selection to query requirements (Liao et al., 2026; Zhao et al., 2026a). Their differences concern both how evidence is represented and when computational work is performed. Our question is whether dependencies established across these stages can support targeted revision after new evidence arrives.

Learned policies and execution-time adaptation. MEM1, MemAgent, and Memory-R1 learn memory updates or utilization policies through reinforcement learning (Zhou et al., 2025; Yu et al., 2025; Yan et al., 2025). MemRL instead learns the utility of episodic memories without updating model weights (Zhang et al., 2026b), while MemGen generates latent memory conditioned on the reasoning state (Zhang et al., 2025a). AdaMEM synthesizes strategies from retrieved trajectories during execution (Zhang et al., 2026c), and MAGE maintains revisable execution-state trees (Chen et al., 2026). These approaches already connect memory to decision-making. We distinguish a procedure's applicability and evidence dependencies from its observed usefulness; neither a high retrieval score nor a successful answer establishes that its factual premises remain valid.

Evaluation and unified analyses. Conversational benchmarks test long-term recall, reasoning, updates, and reflective memory (Maharana et al., 2024; Wu et al., 2025; Tan et al., 2025); incremental interaction and implicit-constraint benchmarks broaden the assessed capabilities (Hu et al., 2026; Li et al., 2026). MemoryArena evaluates memory-dependent multi-session execution, whereas AMA-Bench evaluates understanding of long agent-environment trajectories (He et al., 2026; Zhao et al., 2026b). These settings require distinguishing answer correctness from evidence recovery and successful action. Surveys systematize memory designs and evaluation protocols (Zhang et al., 2025b; Hu et al., 2025), and Wu et al. already provide a modular framework and unified empirical

comparison (Wu et al., 2026). We build on these distinctions, but propose a memory mechanism rather than claim novelty for unified evaluation itself.

Closest overlap and the remaining hypothesis. Human-memory-inspired staging is already explicit in LightMem, and associative binding is central to HippoRAG. Hindsight regenerates synthesized observations when underlying facts change; A-MEM evolves linked representations. TierMem uses source-linked retrieval and verified write-back (Zhu et al., 2026), while TrustMem verifies memory transitions and learns better update behavior (Yang et al., 2026). Continuous consolidation can also degrade useful memory (Zhang et al., 2026a). Consequently, typed storage, source access, feedback, and selective updates are not individually new. The proposed distinction is their coupling through *versioned dependencies across episodic evidence, semantic claims, procedural conditions, and actual use traces*, with fast invalidation followed by bounded background repair. Its benefit over these strong alternatives is a hypothesis requiring a matched comparison, not an established first-of-its-kind claim.

3 Multi-Timescale Memory

Figure 1 summarizes the framework: fast encoding preserves episodes, query-time binding constructs a working context, and slower replay integrates or revises source-backed knowledge.

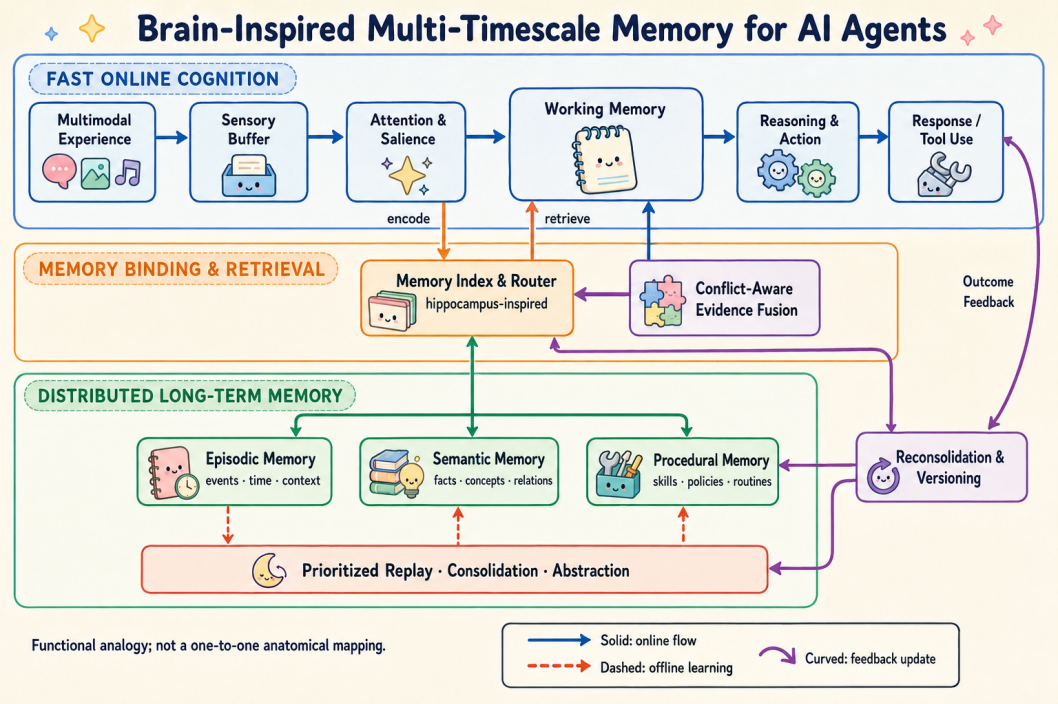


Figure 1: The supplied multi-timescale framework, reproduced unchanged. Brain-region labels express functional inspiration, not anatomical equivalence. The paper formalizes encoding, typed memory, evidence fusion, and versioned reconsolidation; its initial evaluation covers conversational text, not every multimodal or action interface shown.

3.1 State and Functional Organization

At interaction step t, the persistent state is

$$M_t = (\mathcal{E}_t, \mathcal{S}_t, \mathcal{P}_t, \mathcal{G}_t, \mathcal{Q}_t), \qquad W_t = \text{Assemble}(q_t, M_t; B), \tag{1}$$

Timescale	Operation	Committed state change
Fast encoding	Buffer, preserve source, extract cues	Append episode and index; queue abstraction
Query-time binding	Route, retrieve, fuse, respond	Log evidence use; flag supported conflicts
Slow integration	Replay, abstract, validate, revise	Version affected semantic/procedural records

Table 1: Functional interpretation of the supplied framework. Slow integration is a scheduling choice, not a simulation of biological sleep.

where W_t is the working context and B is its memory-token budget. $\mathcal{Q}_t$ holds pending consolidation and repair jobs. The three memory types are logical roles; they need not require three different databases. The language model, embedding model, and verifier are frozen in the proposed initial evaluation. Adaptation changes external memory, not model weights. The working context is the information currently exposed to the reader; the hippocampus-inspired index is a binding mechanism, not a model of hippocampal anatomy. Retrieval-related reconsolidation provides a further functional motivation, but its biological evidence includes specific experimental settings such as fear-memory studies in rats (Nader et al., 2000). That evidence does not establish the validity of our computational design.

Episodic memory: what happened. Each episode $e = (\mathrm{id}, x, a, t_o, t_e, \sigma)$ stores its observed content x, speaker or tool a, observation time t_o, optional event time t_e, and source offsets σ. Records are append-only during normal operation. A correction is a new observation, not an erasure of the original event. An assistant statement is labeled as such and is not treated as an independently verified fact. In the conversational setting, the source establishes what a participant reported, not necessarily objective truth about the world.

Semantic memory: what the evidence supports. A semantic record contains a proposition, entity and relation keys, temporal scope, source-span dependencies, a version, and a status in {active, disputed, superseded, stale}. A derived relation may depend on several episodes. Event time, observation time, and validity interval are separate fields; an absent interval remains unknown. A provenance pointer records the origin of a claim but does not by itself prove entailment.

Procedural memory: how to use experience. A procedure contains an applicability condition, ordered retrieval or action steps, an expected observable check, supporting use traces, and a version. For example, "resolve a demonstrative by locating the earlier recommendation between the same participants" is a retrieval procedure; "the book is *Becoming Nicole*" is not. Learned procedures are candidates until validated on distinct development episodes. Every stored procedure is marked as either a fixed seed rule or an experience-derived rule. This prevents hand-written instructions from being reported as learned skills.

3.2 Fast encoding and typed evidence retrieval

The sensory buffer groups incoming turns into source-preserving episodes. Salience controls indexing and replay priority, not irreversible deletion: low-salience observations remain available as episodes. Named entities, relations, explicit dates, corrections, and unresolved references are indexed. New episodes become retrievable before consolidation finishes. At a session boundary or a fixed buffer threshold, the agent queues related episodes for semantic integration; it does not regenerate its entire memory after each turn.

A bounded router maps a query to entity/relation cues, a requested time scope, and optional procedure preconditions. It retrieves candidates from $\mathcal{E}_t$ and $\mathcal{S}_t$ using the same sparse–dense rank fusion, while $\mathcal{P}_t$ is searched for applicable strategies. The top matching procedure may specify one additional evidence lookup, such as searching for an earlier recommendation. Unmatched preconditions cause the procedure to be skipped, not forced onto the query. This design makes procedure use observable and limits recursive retrieval cost.

Candidate semantic claims are expanded to their recorded episode dependencies. The resulting unit of selection is an *evidence bundle*: a claim together with the source spans needed to interpret it. Bundles share deduplicated sources. They are greedily admitted by query relevance per newly added token under B, subject to their temporal applicability. A required evidence bundle that does not fit is skipped intact rather than silently losing a dependency. The memory budget includes procedure text and metadata exposed to the reader. This is a bounded heuristic, not an optimal retrieval algorithm.

3.3 Conflict-aware evidence fusion

Fusion produces a compact ledger of supported claims, their scopes, and unresolved alternatives before the reader answers. It distinguishes three cases. Complementary records can be joined through a supported relation or coreference link. A later explicit correction to the same scoped attribute can supersede its previous current value. Incompatible records without a justified precedence relation remain disputed. A later timestamp alone does not establish that an unrelated event replaces an earlier one.

For a current-state query, superseded values are not presented as current. For a historical query, the earlier value remains usable within its original scope. A verifier checks the proposed claim against the actual cited spans; when the evidence is ambiguous, the ledger preserves that ambiguity and the reader can abstain. Model verification is fallible and is not a correctness guarantee. In particular, knowing a title from the LLM's parameters is not evidence that a specific participant read that title.

The output is paired with a use trace

$$u_t = (q_t, \mathrm{versions}(D_t), \pi_t, \widehat{y}_t, o_t), \tag{2}$$

where D_t is the used evidence set, π_t is the applied procedure, and o_t is any available external outcome. An absent outcome is recorded as unknown. The trace is a dependency record, not a chain-of-thought transcript.

3.4 Evidence-bound reconsolidation

The dependency graph contains typed `derived-from`, `used-by`, and `supersedes` edges. The first two point from supporting records to their dependents. A supported correction identifies a seed set C_t of directly contradicted or invalidated records. The repair set is

$$A_t = C_t \cup \mathrm{Desc}_{\{\mathrm{derived,used}\}}(C_t; \mathcal{G}_t). \tag{3}$$

Only dependencies on the invalidated scope or exact version propagate the flag. A procedure that merely retrieved a corrected fact is not automatically false; its previous use trace becomes outdated, while the procedure itself is revalidated only if its applicability or validation depends on that fact.

Immediately after identifying a correction, affected derived records are marked stale. They cannot be served as active support while repair waits in the background queue. The original episodes remain retrievable, including the new correction. This separates fast invalidation from expensive rewriting: asynchronous consolidation must not create a window in which already-known stale claims are still treated as valid.

Replay schedules explicit contradictions first, then unsupported-use reports, then ordinary abstraction jobs. Within a class it prioritizes the number of affected active dependents, breaking ties by waiting time. This is an explicit scheduler, not a learned priority policy. Each job reads the relevant source episodes and reconstructs semantic candidates with their dependencies. Successful, externally checked development traces may also yield a generalized procedure; one correct answer is insufficient evidence of transfer.

A revision commits only after source-span checks, temporal-scope checks, and dependency validation. All replacements in a repair transaction receive new versions, and the previous versions remain auditable. If the source snapshot changes before commit, the job is retried against the new snapshot. If a verification check fails or the computation budget is exhausted, the disputed or stale state remains explicit; the system does not restore an unsupported claim simply to finish the update. Unrelated active records are not rewritten. Missing or incorrect dependency edges can nevertheless make a repair incomplete.

Algorithm 1 Multi-timescale operation with evidence-bound reconsolidation

Require: Stream observations x_t, optional queries q_t, memory M_t
Append source-labeled x_t to $\mathcal{E}$; update cues and pending jobs
Identify explicit scoped corrections; mark their dependent records stale
if a query q_t is present **then**
    Route to episodic/semantic candidates and an applicable procedure
    Expand dependencies; pack complete evidence bundles within B
    Fuse by source support and temporal scope; answer or abstain
    Record the used versions, procedure, output, and observed outcome
    Queue evidence-supported conflicts; never use benchmark labels
end if
if a consolidation boundary is reached **then**
    Select jobs by conflict class, dependency impact, and waiting time
    Replay their source episodes; propose abstractions or scoped repairs
    Validate support and dependencies; commit versioned replacements
end if

What the mechanism does and does not establish. Equation 3 guarantees only that the *recorded* dependency closure is considered when the algorithm is implemented correctly. It does not guarantee that all real semantic dependencies were extracted. Likewise, versioning supports traceability, not truth. The research hypothesis is that making these dependencies explicit improves revision locality and future answers enough to justify the additional retrieval and maintenance cost.

4 A Source-Grounded LoCoMo Example

We checked the supplied example against conversation `conv-26` in the LoCoMo release. Turn `D7:11`, dated July 12, 2023, contains Caroline's recommendation of *Becoming Nicole*. Turn `D17:10`, dated October 13, 2023, contains Melanie's report of reading a previously recommended book. The dataset's question asks which book Melanie read following Caroline's suggestion; its reference answer is *Becoming Nicole*, with both turn IDs listed as evidence. The official evaluator identifies category 1 as multi-hop; we retain this released label.

The episodic store preserves both utterances and their dates. Semantic consolidation can propose a recommendation relation grounded in the first turn and a later reading relation whose book argument is initially unresolved. The router identifies the shared participants and recommendation relation, then retrieves the earlier episode. An applicable procedure instructs the agent to check this relation before resolving the demonstrative. Fusion can then propose the title with both source spans attached. If other compatible recommendations remain, it must preserve uncertainty rather than choose a title solely from semantic similarity.

This example reveals three separable failure points: losing the original recommendation during encoding, retrieving the later turn without its antecedent, or asserting an unsupported cross-session link. They require different repairs. Repeating the same answer is not a repair signal, and retrieving a source ID is not proof that its answer-bearing text was available. Figures 1 and 2 show the corresponding architecture and worked example, not measured model performance.

Separating evaluation from adaptation. The "LoCoMo Gold Check" in the supplied illustration belongs strictly to the evaluator. Its result must not be fed into the test-time memory update loop. Memory is built from the native history in chronological order, allowing later observations to trigger reconsolidation during ingestion. Each question then starts from an isolated copy of that history-only snapshot; query-induced changes cannot help later questions. This tests update-aware memory without exposing held-out answers or inventing an additional feedback benchmark.

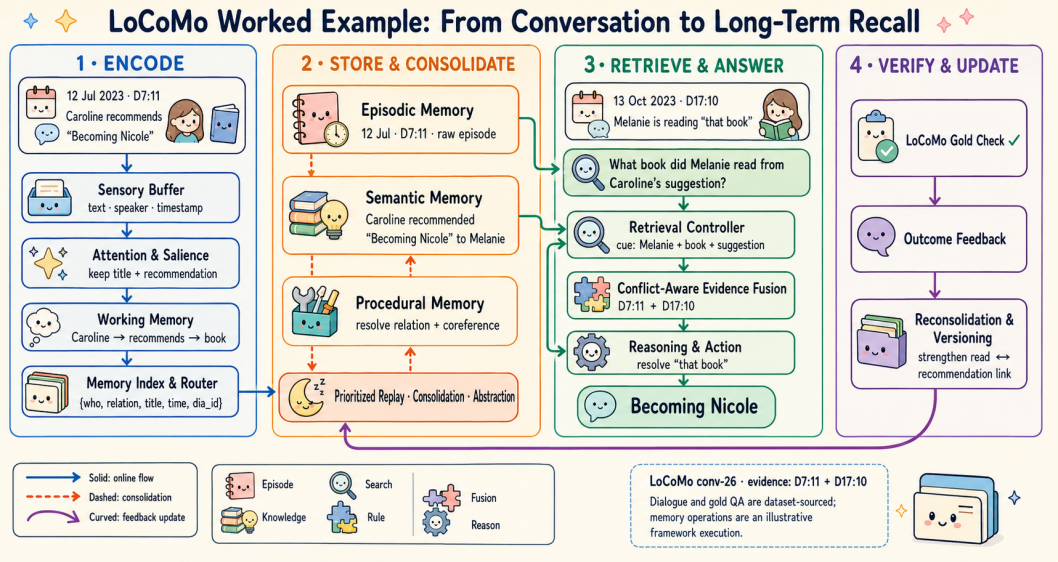


Figure 2: The supplied LoCoMo example, reproduced unchanged. The two turns and reference answer were checked against `conv-26`; memory operations are illustrative, not execution logs. The "Gold Check" branch is *evaluator-only*: neither gold answers nor grading feedback may enter test-time reconsolidation, despite the feedback arrow in the illustration.

5 Experimental Design

We use only LoCoMo and LongMemEval, with three complementary analyses below. All measurements are pending. Table 2 fixes the evaluation scope; no synthetic update benchmark or additional test questions are added. Native images/audio and general tool-use competence are outside this scope.

Benchmark	Evaluation set	Primary score
LoCoMo	10 conversations; 1,540 questions in categories 1–4	Official category-specific F1, averaged over questions
LongMemEval	Cleaned S release; 500 questions	Accuracy under the official question-type and abstention rubrics

Table 2: Two-benchmark protocol (Maharana et al., 2024; Wu et al., 2025). LoCoMo category 5 is excluded explicitly; LongMemEval uses cleaned S, not M, oracle, or V2. Scores are reported separately, never averaged across benchmarks.

Shared protocol. Ingest full conversations chronologically, preserving speakers, dates, and provided text captions. Exclude test questions, answers, evidence labels, answer-session IDs, and answer-bearing-turn flags from indexing and replay; do not ingest released precomputed summaries. Use the supplied question date at inference and reset query-induced changes between questions. Freeze model versions, decoding, prompts, dataset hashes, and the judge configuration before evaluation. Procedures are induced and validated only on independent development dialogues, then frozen before benchmark ingestion.

5.1 Accuracy and Total Cost

Compare MTM with raw-history hybrid RAG, LightMem, HippoRAG 2, and Hindsight. Use their upstream implementations with pinned revisions, not prompt-only imitations. Match the reader, embedding and memory-writing models where supported, preprocessing, and a 4K-token retrieved-

memory budget, including exposed procedure text and metadata. Disclose unsupported controls instead of silently changing a baseline.

Report each benchmark's primary score, online p50/p95 latency, and total model input/output tokens. Separate history construction and replay from query-time routing, retrieval, verification, and answering. Amortize each history's construction over its evaluated questions; count procedure development once and disclose it separately. Include active and archival storage, and keep evaluator-only grading cost separate. The result is one accuracy–cost table, not a large model-by-budget grid.

5.2 Contribution of the core mechanisms

At the same 4K budget, compare the full system with three removals: *no procedural reuse*, using the fixed default router; *no conflict-aware fusion*, passing the same retrieved bundles directly to the reader; and *no evidence-bound reconsolidation*, retaining new episodes and periodic consolidation but disabling dependency-local invalidation and repair. Keep source access, other prompts, session-boundary schedules, and compute ceilings fixed; report realized costs. Use three matched seeds for full and ablated systems and report the mean and variation.

If claiming learned procedures, also compare against a same-budget fixed seed library. An advantage over no procedures alone demonstrates useful procedure access, not learning. These comparisons form one compact ablation table.

5.3 Relational, temporal, and update-aware reasoning

Reuse the same runs and official questions: inspect LoCoMo's multi-hop and temporal categories, and LongMemEval's multi-session, temporal-reasoning, knowledge-update, and abstention subsets. The abstention subset is identified by the release's question-ID suffix, not an invented question type. Compare full and ablated scores to test whether binding and repair help the capabilities they target. Add evidence recall using LoCoMo's annotated turns and LongMemEval's answer sessions where available; score the source text actually exposed to the reader, not merely retrieved IDs. Partial sessions count only when an annotated answer-bearing turn is present. These labels remain evaluator-only. Summarize category effects in one figure.

Report paired 95% bootstrap intervals grouped by conversation or independent history, with seeds matched across ablations. LoCoMo has only ten independent conversations, limiting uncertainty estimates. Use every item in the declared sets; retain failed calls in the denominator and report their count and retry policy. Native knowledge-update questions test revision during history ingestion; they do not by themselves establish post-answer lifelong learning or complete dependency-repair correctness.

6 Limitations and Research Risks

The framework is presently an algorithmic proposal, not an empirically validated system. Its hardest component is reliable dependency extraction: an incorrect link can create a false abstraction, while a missing link can prevent the necessary repair. Source-grounded verification can still fail, and repeated model agreement is not independent confirmation. Temporal interpretation and conflicting user reports may remain unresolved.

Procedural memory also introduces an attribution problem. A useful fixed retrieval instruction is not evidence that the system learned a reusable strategy from experience. The specified seed-rule control and held-out procedure validation are therefore necessary. QA success would establish conversational utility, not general action learning or multimodal cognition.

Typed stores, provenance, feedback, and sleep-time updates all have substantial precedent. The proposed coupling may offer little beyond a strong existing system, or its extra verification may outweigh the accuracy benefit. Negative results would restrict the framework's useful operating conditions; they should not be obscured by changing budgets or omitting background costs. Finally, append-only episodic storage raises privacy and storage concerns. Deployment needs authorized deletion and dependent-record removal; research versioning must not be presented as a reason to retain personal data forever.

7 Conclusion

We propose a brain-inspired memory framework in which episodes provide evidence, semantic memory captures scoped knowledge, and procedural memory guides evidence use. Its central idea is to reuse the dependencies created during abstraction and retrieval for targeted, versioned reconsolidation. The verified LoCoMo case motivates relational binding; evaluation on LoCoMo and LongMemEval separates accuracy and cost, component effects, and relational, temporal, and update-aware reasoning. Whether this coupling improves long-term agent memory at an acceptable cost is the empirical question the proposed experiments must answer.

AI Use Statement

Generative AI assisted with literature inspection, framework formalization, experimental-design refinement, manuscript drafting, and LaTeX preparation. No benchmark measurements were generated or inferred in preparing this draft. Author review of the scientific claims, references, and final disclosure is required before submission. The origin and editing history of supplied illustrations must also be confirmed by the authors for the final disclosure.

Reproducibility Statement

The algorithm, budget rules, and evaluation boundaries are specified in Sections 3–5, with both supplied conceptual figures in the main text and implementation contracts in the appendix. An executable end-to-end implementation, run manifests, and benchmark outputs are still required. The worked example is a dataset check, not a reproduced system experiment.

References

Yaoqi Chen, Haibin Lai, Yuru Feng, Chuyu Han, Qianxi Zhang, Baotong Lu, Menghao Li, Xinjiang Wang, Zhirui Wang, Shusen Xu, Zengzhong Li, Zewen Jin, Hao Wu, Cheng Li, and Qi Chen. Beyond Semantic Organization: Memory as Execution State Management for Long-Horizon Agents, 2026. URL https://arxiv.org/abs/2606.06090.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory, 2025. URL https://arxiv.org/abs/2504.19413.

Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. LightMem: Lightweight and Efficient Memory-Augmented Generation. In *International Conference on Learning Representations*, volume 2026, pp. 98706–98729, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/a05b72653ec5b473732129829ae04195-Paper-Conference.pdf.

Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *Advances in Neural Information Processing Systems*, volume 37, pp. 59532–59569. Curran Associates, Inc., 2024. doi: 10.52202/079017-1902. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/6ddc001d07ca4f319af96a3024f6dbd1-Paper-Conference.pdf.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 21497–21515. PMLR, 2025. URL https://proceedings.mlr.press/v267/gutierrez25a.html.

Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks, 2026. URL https://arxiv.org/abs/2602.16313.

Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2507.05257.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the Age of AI Agents, 2025. URL https://arxiv.org/abs/2512.13564.

Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. MAGMA: A Multi-Graph based Agentic Memory Architecture for AI Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 36848–36865. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1709. URL https://aclanthology.org/2026.acl-long.1709/.

Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI Agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 25961–25970. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1318. URL https://aclanthology.org/2025.emnlp-main.1318/.

Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016. doi: 10.1016/j.tics.2016.05.004. URL https://pubmed.ncbi.nlm.nih.gov/27315762/.

Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects, 2025. URL https://arxiv.org/abs/2512.12818.

Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A Human-Inspired Reading Agent with Gist Memory of Very Long Contexts. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 26396–26415. PMLR, 2024. URL https://proceedings.mlr.press/v235/lee24c.html.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, pp. 9459–9474. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

Yifei Li, Weidong Guo, Lingling Zhang, Rongman Xu, Muye Huang, Hui Liu, Lijiao Xu, Yu Xu, and Jun Liu. Locomo-Plus: Beyond-Factual Cognitive Memory Evaluation Framework for LLM Agents, 2026. URL https://arxiv.org/abs/2602.10715.

Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhen Tao, Huayi Lai, Hao Wu, Bo Tang, Zhengren Wang, Zhaoxin Fan, Ningyu Zhang, Linfeng Zhang, Junchi Yan, Mingchuan Yang, Tong Xu, Wei Xu, Huajun Chen, Haofen Wang, Hongkang Yang, Wentao Zhang, Zhi-Qin John Xu, Siheng Chen, and Feiyu Xiong. MemOS: A Memory OS for AI System, 2025. URL https://arxiv.org/abs/2507.03724.

Yuxin Liao, Le Wu, Min Hou, Hao Liu, Han Wu, and Zishu Wang. LeanMem: Simple and Efficient Long-Term Memory for LLM Agents, 2026. URL https://arxiv.org/abs/2608.03463.

Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient Lifelong Memory for LLM Agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2601.02553.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.747. URL https://aclanthology.org/2024.acl-long.747/.

James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. doi: 10.1037/0033-295X.102.3.419. URL https://pubmed.ncbi.nlm.nih.gov/7624455/.

Karim Nader, Glenn E. Schafe, and Joseph E. Le Doux. Fear memories require protein synthesis in the amygdala for reconsolidation after retrieval. *Nature*, 406:722–726, 2000. doi: 10.1038/35021052. URL https://www.nature.com/articles/35021052.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems, 2023. URL https://arxiv.org/abs/2310.08560.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–22. ACM, 2023. doi: 10.1145/3586183.3606763. URL http://dx.doi.org/10.1145/3586183.3606763.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A Temporal Knowledge Graph Architecture for Agent Memory, 2025. URL https://arxiv.org/abs/2501.13956.

Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.14052.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 8634–8652. Curran Associates, Inc., 2023. doi: 10.52202/075280-0377. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf.

Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 19336–19352. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.989. URL https://aclanthology.org/2025.findings-acl.989/.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2305.16291.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.10813.

Yanchen Wu, Tenghui Lin, Yingli Zhou, Fangyuan Zhang, Qintian Guo, Xun Zhou, Sibo Wang, Xilin Liu, Yuchi Ma, and Yixiang Fang. Memory in the LLM Era: Modular Architectures and Strategies in a Unified Framework, 2026. URL https://arxiv.org/abs/2604.01707.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 17577–17604. Curran Associates, Inc., 2025. doi: 10.52202/085713-0593. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/19909c36f51abc4856b4560aff3d36d6-Paper-Conference.pdf.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning, 2025. URL https://arxiv.org/abs/2508.19828.

Tianyu Yang, Sudipta Paul, Vijay Srinivasan, Vivek Kulkarni, and Srinivas Chappidi. TRUSTMEM: Learning trustworthy memory consolidation for LLM agents with long-term memory, 2026. URL https://arxiv.org/abs/2606.25161.

Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and Hao Zhou. MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent, 2025. URL https://arxiv.org/abs/2507.02259.

Dylan Zhang, Yanshan Lin, Zhengkun Wu, Yihang Sun, Bingxuan Li, Dianqi Li, and Hao Peng. Useful memories become faulty when continuously updated by LLMs, 2026a. URL https://arxiv.org/abs/2605.12978. Version 2, August 29, 2026.

Guibin Zhang, Muxin Fu, and Shuicheng Yan. MemGen: Weaving Generative Latent Memory for Self-Evolving Agents, 2025a. URL https://arxiv.org/abs/2509.24704.

Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory, 2026b. URL https://arxiv.org/abs/2601.03192.

Yunxiang Zhang, Yiheng Li, Ali Payani, and Lu Wang. AdaMEM: Test-Time Adaptive Memory for Language Agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026c. URL https://arxiv.org/abs/2606.05684.

Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A Survey on the Memory Mechanism of Large Language Model-based Agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025b. doi: 10.1145/3748302. URL http://dx.doi.org/10.1145/3748302.

Beidi Zhao, Yaoqi Chen, Yuru Feng, Menghao Li, Qianxi Zhang, Baotong Lu, Jianan Lu, Zhirui Wang, Xinjiang Wang, Shusen Xu, Zengzhong Li, Xiaoxiao Li, and Qi Chen. MESA:Task-Adaptive Multi-Structure Evidence Selection for Long-Horizon Agent Memory, 2026a. URL https://arxiv.org/abs/2608.10108.

Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, Yuandong Tian, and Jishen Zhao. AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026b. URL https://arxiv.org/abs/2602.22769.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19724–19731, 2024. doi: 10.1609/aaai.v38i17.29946. URL http://dx.doi.org/10.1609/aaai.v38i17.29946.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents, 2025. URL https://arxiv.org/abs/2506.15841.

Qiming Zhu, Shunian Chen, Rui Yu, Zhehao Wu, and Benyou Wang. From lossy to verified: A provenance-aware tiered memory for agents, 2026. URL https://arxiv.org/abs/2602.17913.

A Implementation and Evaluation Contracts

A minimal implementation. Represent episodic records, semantic claims, procedures, use traces, and version edges in a local transactional store. Build a shared sparse–dense index with type filters; rank fusion must be identical across ablations. The initial router is a frozen model call returning entity keys, relation keys, temporal scope, and at most one applicable procedure. Fix a maximum of two retrieval rounds: initial retrieval and one procedure-requested lookup. Use a source verifier that returns supported, contradicted, or unresolved, with exact source offsets. Treat parser failure as a system failure, not silently as abstention or an incorrect answer.

Record and transaction invariants. Every active semantic record has at least one resolvable episode-span dependency. Every derived edge identifies the source version and scope. A use trace stores the exact versions read. Commit a repair only against the source snapshot from which it was constructed. Mark known-stale descendants unavailable before scheduling a rebuild. Historical applicability and current applicability are checked separately. A failed repair never silently reactivates a stale claim. These invariants check software behavior; they do not validate the semantic correctness of extracted claims.

Procedure induction. Start with a small declared seed library shared by full and control systems. Use independently generated development dialogues, split by source-template family before generating paraphrases, with no benchmark passages, questions, or answers. Fix the generation templates and procedure-selection criteria before benchmark runs and record the full development-generation cost. Generate candidate procedures only from source-grounded development use traces or externally verified training interactions. Abstract entity names and answer strings out of the procedure. Validate its preconditions and observable check on distinct held-out development episodes, all separated from benchmark histories. Store the validation trace IDs and retire a procedure when its supporting conditions are invalidated. The benchmark test set cannot supply demonstrations, correction labels, or induction targets. No claim of learned procedural memory is warranted until this pipeline and its fixed-rule control have been evaluated.

Replay and scheduling. Use session boundaries for ordinary consolidation and a fixed compute cap per boundary for every compared method. Explicit scoped contradictions create immediate invalidation jobs. Queue ordering is lexicographic: conflict class, number of active dependents, then elapsed queue time. Record queue length, actual jobs completed, and source tokens examined. When support cannot be recovered within the budget, leave the item disputed and record the deferred work. Report any global graph scan separately from dependency-local repair time; a local update set does not prove local search cost.

History-only adaptation and isolated QA. Construct memory from chronologically revealed benchmark conversations; replay may use only observations already seen. Use identical boundary budgets for full and no-reconsolidation variants. Clone the final history-only state for each official question, preserve its question date, and discard all query-induced changes after answering. Keep gold answers, annotated source IDs, and answer-bearing flags in a separate evaluator. LongMemEval abstention uses the official question-ID mask; it is not a seventh question type. No generated corrections, additional test questions, or separate longitudinal benchmark enter the reported scores.

Required evidence before submission. The new framework still requires an integrated implementation; validated procedure induction; official-baseline runs; full LoCoMo and LongMemEval results; and the three matched comparisons in Section 5. A runnable module, a conceptual figure, or a successful unit test alone does not establish any of these empirical claims. No result table is populated in this draft because these measurements have not been supplied.